Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №5
на тему
«ХЕШ-ФУНКЦИИ.
ГОСТ Р 34.11-2012 И SHA-1»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Краткие теоретические сведения 4
2.1 Криптографическая хеш-функция 4
2.2 ГОСТ Р 34.11-2012 4
2.3 SHA-1 5
2.4 Применение 5
3 Выполнение работы 6
3.1 Структура программного средства 6
3.2 Схема вычисления хеш-функции 6
3.3 Реализация ГОСТ Р 34.11-2012 6
3.4 Реализация SHA-1 7
3.5 Интерфейс 7
3.6 Тестирование 8
3.7 Сравнение алгоритмов 10
Заключение 11
Список использованных источников 12
Приложение А (Обязательное) Листинг программного кода 13

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение криптографических хеш-функций и получение практических навыков контроля целостности сообщений.

В ходе работы требовалось реализовать без готовых криптографических библиотек программное средство контроля целостности на основе двух хеш-функций:

1 ГОСТ Р 34.11-2012 («Стрибог») с длиной кода 512 и 256 бит;

2 SHA-1 с длиной кода 160 бит.

Необходимо было реализовать сами преобразования стандартов, встроить хеширование в терминальный интерфейс программы, проверить реализацию эталонными примерами RFC 6986 и FIPS 180-4 и исследовать лавинный эффект.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Криптографическая хеш-функция

Хеш-функция отображает сообщение произвольной длины в значение фиксированной длины – дайджест. Криптографическая хеш-функция должна обладать тремя свойствами [1]:

– стойкость к нахождению прообраза: по дайджесту h трудно найти сообщение M, для которого $H(M) = h$;

– стойкость ко второму прообразу: по сообщению M трудно найти другое M', для которого $H(M') = H(M)$;

– стойкость к коллизиям: трудно найти два разных сообщения с одинаковым дайджестом.

К этим свойствам добавляется лавинный эффект: изменение одного бита входа должно менять примерно половину битов выхода. Контроль целостности основан на том, что любое искажение данных с большой вероятностью изменяет дайджест, а восстановить исходные данные по дайджесту нельзя.

2.2 ГОСТ Р 34.11-2012

Хеш-функция работает с 512-битным состоянием h, 512-битным счётчиком длины N и 512-битной контрольной суммой Σ. Для 512-битного варианта начальное значение $h = 0^{512}$, для 256-битного – $h = (00000001)^{64}$ [2].

Основу составляет функция сжатия

$$g_N(h, m) = E(\mathrm{LPS}(h \oplus N), m) \oplus h \oplus m, \tag{2.1}$$

где m – очередной блок сообщения. Функция $E(K, m)$ выполняет 12 раундов вида «LPS» с ключами $K_1, \ldots, K_{13}$. Первый ключ равен K, а каждый следующий получается по расписанию

$$K_{i+1} = \mathrm{LPS}(K_i \oplus C_i). \tag{2.2}$$

Преобразования S, P и L действуют на 512-битный вектор, разбитый на 64 байта: S заменяет каждый байт по таблице Π, P переставляет байты по таблице «Tau», а L умножает восемь 64-битных частей на матрицу A над полем $\mathrm{GF}(2)$.

Сообщение обрабатывается с конца: из него выделяются последние 512 бит, пока длина не станет меньше 512. Остаток дополняется по правилу $m = 0^{511-|M|} \parallel 1 \parallel M$. После этого выполняются две заключительные функции сжатия $g_0(h, N)$ и $g_0(h, \Sigma)$. Для 256-битного варианта из результата берутся старшие 256 бит.

2.3 SHA-1

SHA-1 построена по схеме Меркла–Дамгарда [3]. Сообщение дополняется битом 1, нулями и 64-битной длиной в битах до кратности 512, после чего каждый блок обрабатывается 160-битным буфером A, B, C, D, E. В 80 раундах используются четыре константы K_t, три логические функции и расширение слов

$$W_t = W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3} \tag{2.3}$$

с циклическим сдвигом влево на один бит. Дайджест является конкатенацией пяти слов буфера.

SHA-1 считается устаревшей: практические коллизии были построены в 2017 году, поэтому алгоритм непригоден для новых систем электронной подписи. В учебной работе он интересен как простой и хорошо документированный пример хеш-функции.

2.4 Применение

Хеш-функции применяются для контроля целостности файлов и сообщений, в электронной подписи (подписывается дайджест, а не всё сообщение), в протоколах аутентификации и в схемах хеширования паролей. ГОСТ Р 34.11-2012 входит в набор российских криптографических стандартов и используется вместе с ГОСТ Р 34.10-2012.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Хеш-функции реализованы на языке Go в пакете *internal/crypto/hash* и разделены на три файла:

– *gost3411.go* – преобразования ГОСТ Р 34.11-2012, функция сжатия и финализация;

– *sha1.go* – SHA-1 по FIPS 180-4;

– *hash.go* – реестр алгоритмов с общим интерфейсом *hash.Hash* стандартной библиотеки.

Интерфейс программы получает алгоритмы из реестра и поэтому работает с обеими реализациями одинаково. Новая вкладка *Hash* построена в пакете *internal/ui/hashpane* по образцу асимметричной панели и зарегистрирована в *main.go*.

3.2 Схема вычисления хеш-функции

Последовательность обработки сообщения по ГОСТ Р 34.11-2012 показана на рисунке 3.1.

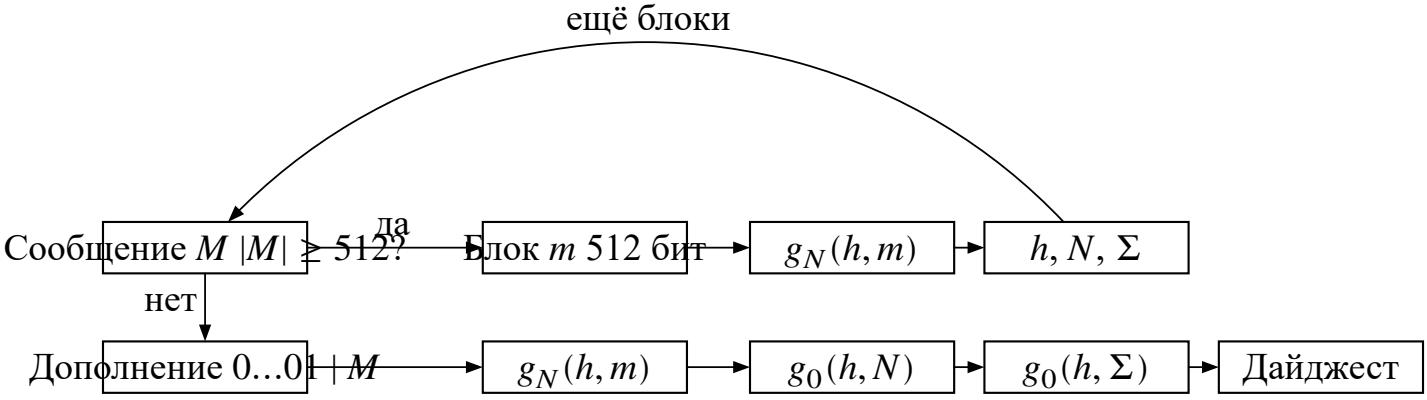


Рисунок 3.1 – Схема вычисления хеш-функции ГОСТ Р 34.11-2012

3.3 Реализация ГОСТ Р 34.11-2012

Состояние хранится массивом из 64 байт «младшим байтом вперёд»: элемент с индексом 0 соответствует a_0 из записи $a_{63} \parallel \ldots \parallel a_0$. Блок сообщения записывается в состояние в естественном порядке, а дайджест перед выводом переворачивается: стандарт записывает вектор старшим байтом вперёд, поэтому эталонные дайджесты RFC 6986 получаются именно в такой записи. Это же соглашение применяется к сообщениям из примеров: вектор M1 из п. 10.1 читается как 63-байтовая строка цифр, а его дополненный блок совпадает с `01 || M1`.

Каждое преобразование оформлено отдельной функцией с формулой из стандарта в комментарии:

– *S* заменяет байты по таблице *pi* из п. 6.2;
– *P* переставляет байты по таблице *tau* из п. 6.3;
– *L* умножает каждую 64-битную часть на матрицу *aMatrix* из п. 6.4;
– *keySchedule* вычисляет $K_{i+1} = \mathrm{LPS}(K_i \oplus C_i)$;
– *E* выполняет 12 раундов, *gN* – функцию сжатия g_N.

Константы П, «Tau», *A* и $C_1 \ldots C_{12}$ перенесены из RFC 6986, п. 6.2–6.5. Реализация использует только стандартную библиотеку: массивы байтов, сдвиги и сложение по модулю 2^{512} выполняются вручную.

Стандарт обрабатывает сообщение с конца, поэтому принятые через *Write* байты накапливаются и хешируются в *Sum*. Для терминального средства контроля целостности это приемлемо; память занята всем сообщением, зато порядок блоков и дополнение точно соответствуют стандарту. Функция *Sum* не изменяет состояние, а *Reset* очищает накопленное сообщение.

3.4 Реализация SHA-1

SHA-1 реализована потоково: *Write* дополняет 64-байтный буфер и обрабатывает полные блоки сразу, а *Sum* работает с копией состояния. Такой подход не требует хранить всё сообщение и совпадает с контрактом *hash.Hash*.

Функция *finish* добавляет бит 1, нули и 64-битную длину в битах. В *processBlock* первые 16 слов читаются из блока в порядке старшего байта, остальные 64 слова получаются по формуле расширения. В каждом раунде вычисляется логическая функция, прибавляются константа, слово сообщения и результаты сдвигов; после 80 раундов буфер складывается со значением до блока по модулю 2^{32}.

3.5 Интерфейс

Вкладка *Hash* состоит из трёх полос. Верхняя полоса показывает выбранный алгоритм, размер дайджеста и блока; *Enter* переключает алгоритм по кругу. Средняя полоса содержит поле ввода сообщения слева и панель результата справа: дайджест в шестнадцатеричном виде, поле ожидаемого значения и строку вердикта *СОВПАДАЕТ* или *НЕ СОВПАДАЕТ*. Нижняя полоса содержит источник данных (ручной ввод или файл), кнопки *HASH*, *AVALANCHE*, *SAVE TO FILE* и строку состояния.

Клавиша *Ctrl+D* переносит вычисленный дайджест в поле ожидаемого значения, что позволяет сразу увидеть вердикт; *Alt+D* очищает ввод. Кнопка *AVALANCHE* инвертирует младший бит первого байта сообщения, хеширует оба варианта и выводит число изменившихся битов дайджеста. Ввод можно взять из файла через встроенный выбор файлов; дайджест сохраняется в текстовый файл.

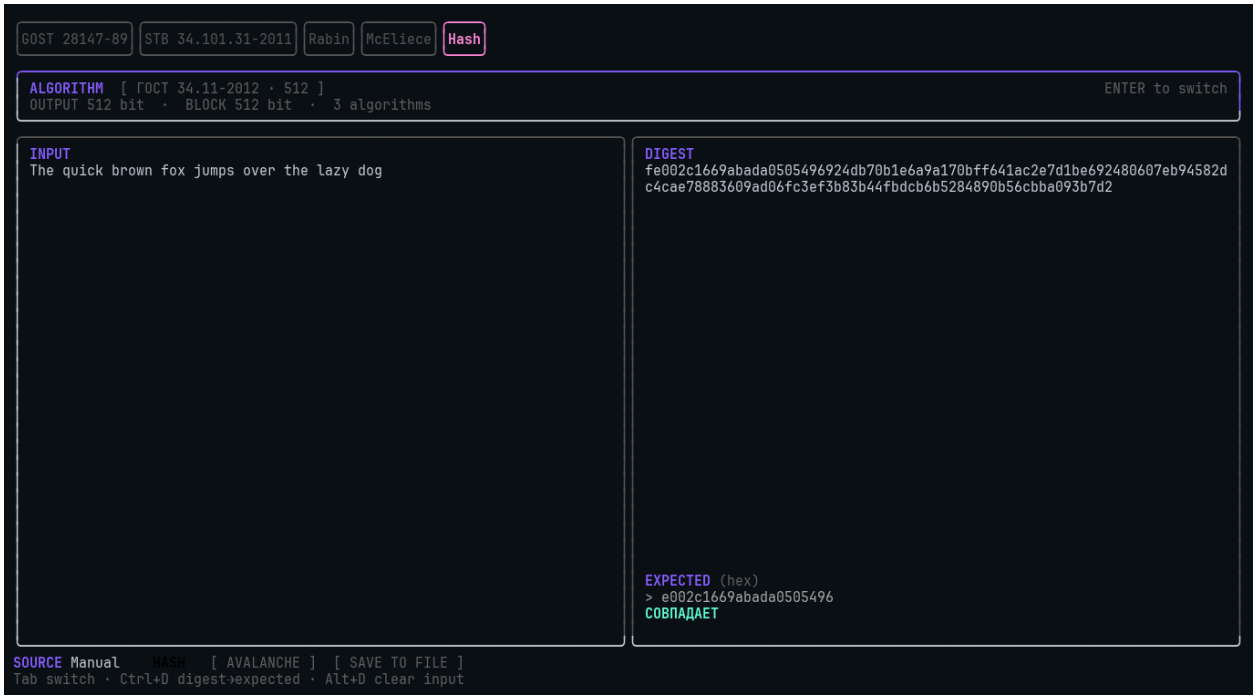


Рисунок 3.2 – Вкладка *Hash*: дайджест ГОСТ 34.11-2012 (512 бит) и вердикт сравнения с ожидаемым значением

3.6 Тестирование

Корректность проверена командой `go test ./internal/crypto/hash`. Все тесты проходят.

TestGOST3411Example1Intermediates воспроизводит промежуточные значения примера 1 из RFC 6986: подстановку $S(0^{512})$, перестановку $P(S(0^{512}))$, ключ $K = \mathrm{LPS}(0^{512})$, а также значения на выходах раундов и результаты функций сжатия. Это позволяет локализовать ошибку на конкретном преобразовании, а не только увидеть неверный итоговый дайджест.

TestGOST3411RFCExamples проверяет четыре эталонных дайджеста примера 1 и примера 2 из п. 10 RFC 6986 (таблица 3.1).

Таблица 3.1 – Эталонные дайджесты ГОСТ Р 34.11-2012 из RFC 6986

Алгоритм	Вход	Ожидаемый дайджест	Результат
ГОСТ 512	M1, 63 Б	486f64c1917879417fef082b3381a4e2 11c324f074654c38823a7b76f830ad00 fa1fbae42b1285c0352f227524bc9ab1 6254288dd6863dccd5b9f54a1ad0541b	совпадает
ГОСТ 256	M1, 63 Б	00557be5e584fd52a449b16b0251d05d 27f94ab76cbaa6da890b59d8ef1e159d	совпадает
ГОСТ 512	M2, 72 Б	28fbc9bada033b1460642bdcddb90c3f b3e56c497ccd0f62b8a2ad4935e85f03	совпадает

Алгоритм	Вход	Ожидаемый дайджест	Результат
		`7613966de4ee00531ae60f3b5a47f8da` `e06915d5f2f194996fcabf2622e6881e`	
ГОСТ 256	M2, 72 Б	`508f7e553c06501d749a66fc28c6cac0` `b005746d97537fa85d9e40904efed29d`	совпал

Тест сравнивает полные строки, поэтому столбец «Результат» означает побайтное совпадение полученного и ожидаемого значений. Сообщения M1 и M2 заданы в RFC 6986 в шестнадцатеричном виде и разбираются тестом напрямую.

Запись дайджеста. Состояние и дайджест выводятся в нотации RFC 6986, п. 10, – старшим байтом вперёд. Именно поэтому четыре нормативных дайджеста совпадают со стандартом буквально. Утилиты, такие как `rhash`, печатают те же байты в обратном порядке, то есть `rhash(M) = reverse(H(M))`. Например, для пустого сообщения реализация даёт `8a1a1c4c...a25d948e`, а `printf '' | rhash --gost12-512` – `8e945da2...f4c1c1a8a`. Обращением второго значения получается первое. То же соотношение выполняется для M1 и M2: обращением дайджеста `486f64c1...` получается `1b54d01a...`, который `rhash` выдаёт для тех же байтов сообщения. По той же причине шестнадцатеричные сообщения в §10 записаны старшим байтом вперёд: M1 из 63 байт – это строка цифр «0123…», а не последовательность `3231...`, показанная в тексте RFC.

Для SHA-1 проверены примеры FIPS 180-4: пустое сообщение, строка `abc`, сообщение длиной 448 бит и строка из миллиона символов `a` (таблица 3.2).

Таблица 3.2 – Эталонные дайджесты SHA-1 из FIPS 180-4

Вход	Ожидаемый дайджест	Результат
пустое сообщение	`da39a3ee5e6b4b0d3255bfef95601890` `afd80709`	совпал
`abc`	`a9993e364706816aba3e25717850c26c` `9cd0d89d`	совпал
448 бит	`84983e441c3bd26ebaae4aa1f95129e5` `e54670f1`	совпал
миллион `a`	`34aa973cd4c4daa4f61eeb2bdbad2731` `6534016f`	совпал

Дополнительно *TestSHA1MatchesStandardLibrary* сверяет реализацию со стандартной библиотекой *crypto/sha1* на детерминированных псевдослучайных сообщениях длиной от 0 до 700 байт. *TestGOST3411Incremental* и *TestSHA1Incremental* проверяют, что хеширование порциями разной длины (1, 3, 7, 63, 64, 65 байт) даёт тот же дайджест, что и хеширование всего сообщения сразу. *TestGOST3411Reset* проверяет возврат в начальное состояние.

Лавинный эффект измерен тестом *TestAvalanche*. Для фиксированного 64-байтного сообщения по очереди инвертировался каждый из 512 входных битов и подсчитывалось число изменившихся битов выхода (таблица 3.3).

Таблица 3.3 – Лавинный эффект: число изменившихся битов при инверсии одного входного бита

Алгоритм	Выход, бит	Минимум	Максимум	Среднее
ГОСТ 512	512	212	289	256,3 (50,1 %)
ГОСТ 256	256	103	152	127,8 (49,9 %)
SHA-1	160	59	101	79,9 (49,9 %)

Все значения попадают в допустимый диапазон 30–70 %, то есть ни один входной бит не оказывает локального влияния на выход.

3.7 Сравнение алгоритмов

Скорость измерена тестом *BenchmarkAlgorithms* на сообщении длиной 1 МиБ (таблица 3.4). Реализации написаны для читаемости, без табличных оптимизаций преобразований S, P и L.

Таблица 3.4 – Сравнение реализованных хеш-функций

Алгоритм	Дайджест, бит	Блок, бит	Скорость
ГОСТ Р 34.11-2012 · 512	512	512	1,14 МБ/с
ГОСТ Р 34.11-2012 · 256	256	512	1,15 МБ/с
SHA-1	160	512	272,8 МБ/с

ГОСТ Р 34.11-2012 даёт вдвое больший дайджест и работает медленнее: каждый блок требует 12 раундов с байтовой подстановкой, перестановкой и умножением на матрицу, тогда как SHA-1 выполняет 80 раундов над 32-битными словами. Разница в скорости в основном объясняется отсутствием таблиц предвычислений в учебной реализации ГОСТ.

ЗАКЛЮЧЕНИЕ

В ходе работы реализованы две криптографические хеш-функции: ГОСТ Р 34.11-2012 с длиной кода 512 и 256 бит и SHA-1. Обе реализации написаны вручную, используют только стандартную библиотеку и оформлены отдельными функциями для каждого преобразования стандарта. Корректность подтверждена эталонными примерами RFC 6986 и FIPS 180-4, включая промежуточные значения преобразований, а также тестами на порционное хеширование и сверкой SHA-1 со стандартной библиотекой.

Хеш-функции встроены в терминальный интерфейс отдельной вкладкой *Hash*: она позволяет выбрать алгоритм, ввести сообщение или прочитать файл, сравнить дайджест с ожидаемым значением и оценить лавинный эффект. Измерения показали, что инверсия одного входного бита меняет около половины битов выхода для всех трёх алгоритмов.

Ограничением учебной реализации ГОСТ является скорость: отсутствие табличных оптимизаций делает её примерно в 240 раз медленнее SHA-1 на больших сообщениях. Для контроля целостности небольших файлов этого достаточно, для потоковой обработки потребовались бы предвычисления.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Menezes, A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press, 1996. – 780 с.

[2] Dolmatov, V. GOST R 34.11-2012: Hash Function / V. Dolmatov, A. Degtyarev. – 2013.

[3] Technology. Secure Hash Standard (SHS) / National Institute of Standards and Technology. – 2015.

[4] Rabin, M.O. Digitalized Signatures and Public-Key Functions as Intractable as Factorization / M.O. Rabin. – 1979.

[5] Лабораторные работы по дисциплине «Методы защиты информации» : методические указания. – 2024.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – ГОСТ Р 34.11-2012

```
package hash

// Хеш-функция ГОСТ Р 34.11-2012 («Стрибог»).
//
// Реализация следует RFC 6986. Состояние хеш-функции — вектор из 512 бит,
// то есть 64 байта. Внутри он хранится «младшим байтом вперёд»: элемент
// state[i] соответствует a_i из записи a_63 || ... || a_0. Поэтому при
// выводе дайджеста и при разборе эталонных примеров байты переставляются
// в обратном порядке: в стандарте вектор записывается старшим байтом вперёд.

import (
  "encoding/binary"
  "encoding/hex"
  "hash"
)

const (
  gostBlockSize = 64
  gost512Size   = 64
  gost256Size   = 32
)

// Pi — нелинейная подстановка Pi' из RFC 6986, п. 6.2. Каждый байт вектора
// заменяется по таблице: S(a) = Pi(a_63) || ... || Pi(a_0).
var pi = [256]byte{
  252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250,
  218, 35, 197, 4, 77, 233, 119, 240, 219, 147, 46,
  153, 186, 23, 54, 241, 187, 20, 205, 95, 193, 249,
  24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66,
  139, 1, 142, 79, 5, 132, 2, 174, 227, 106, 143,
  160, 6, 11, 237, 152, 127, 212, 211, 31, 235, 52,
  44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253,
  58, 206, 204, 181, 112, 14, 86, 8, 12, 118, 18,
  191, 114, 19, 71, 156, 183, 93, 135, 21, 161, 150,
  41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158,
  178, 177, 50, 117, 25, 61, 255, 53, 138, 126, 109,
  84, 198, 128, 195, 189, 13, 87, 223, 245, 36, 169,
  62, 168, 67, 201, 215, 121, 214, 246, 124, 34, 185,
  3, 224, 15, 236, 222, 122, 148, 176, 188, 220, 232,
  40, 80, 78, 51, 10, 74, 167, 151, 96, 115, 30,
  0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65,
  173, 69, 70, 146, 39, 94, 85, 47, 140, 163, 165,
  125, 105, 213, 149, 59, 7, 88, 179, 64, 134, 172,
  29, 247, 48, 55, 107, 228, 136, 217, 231, 137, 225,
  27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144,
  202, 216, 133, 97, 32, 113, 103, 164, 45, 43, 9,
  91, 203, 155, 37, 208, 190, 229, 108, 82, 89, 166,
  116, 210, 230, 244, 180, 192, 209, 102, 175, 194, 57,
  75, 99, 182,
}

// Tau — перестановка байтов из RFC 6986, п. 6.3:
// P(a) = a_{Tau(63)} || ... || a_{Tau(0)}.
var tau = [64]byte{
  0, 8, 16, 24, 32, 40, 48, 56,
  1, 9, 17, 25, 33, 41, 49, 57,
```

```
  2, 10, 18, 26, 34, 42, 50, 58,
  3, 11, 19, 27, 35, 43, 51, 59,
  4, 12, 20, 28, 36, 44, 52, 60,
  5, 13, 21, 29, 37, 45, 53, 61,
  6, 14, 22, 30, 38, 46, 54, 62,
  7, 15, 23, 31, 39, 47, 55, 63,
}

// aMatrix — строки матрицы A линейного преобразования l (RFC 6986, п. 6.4).
// Строка j используется при умножении на бит b_i, где i = 63 - j.
var aMatrix = [64]uint64{
          0x8e20faa72ba0b470,     0x47107ddd9b505a38,     0xad08b0e0c3282d1c,
0xd8045870ef14980e,
          0x6c022c38f90a4c07,     0x3601161cf205268d,     0x1b8e0b0e798c13c8,
0x83478b07b2468764,
          0xa011d380818e8f40,     0x5086e740ce47c920,     0x2843fd2067adea10,
0x14aff010bdd87508,
          0x0ad97808d06cb404,     0x05e23c0468365a02,     0x8c711e02341b2d01,
0x46b60f011a83988e,
          0x90dab52a387ae76f,     0x486dd4151c3dfdb9,     0x24b86a840e90f0d2,
0x125c354207487869,
          0x092e94218d243cba,     0x8a174a9ec8121e5d,     0x4585254f64090fa0,
0xaccc9ca9328a8950,
          0x9d4df05d5f661451,     0xc0a878a0a1330aa6,     0x60543c50de970553,
0x302a1e286fc58ca7,
          0x18150f14b9ec46dd,     0x0c84890ad27623e0,     0x0642ca05693b9f70,
0x0321658cba93c138,
          0x86275df09ce8aaa8,     0x439da0784e745554,     0xafc0503c273aa42a,
0xd960281e9d1d5215,
          0xe230140fc0802984,     0x71180a8960409a42,     0xb60c05ca30204d21,
0x5b068c651810a89e,
          0x456c34887a3805b9,     0xac361a443d1c8cd2,     0x561b0d22900e4669,
0x2b838811480723ba,
          0x9bcf4486248d9f5d,     0xc3e9224312c8c1a0,     0xeffa11af0964ee50,
0xf97d86d98a327728,
          0xe4fa2054a80b329c,     0x727d102a548b194e,     0x39b008152acb8227,
0x9258048415eb419d,
          0x492c024284fbaec0,     0xaa16012142f35760,     0x550b8e9e21f7a530,
0xa48b474f9ef5dc18,
          0x70a6a56e2440598e,     0x3853dc371220a247,     0x1ca76e95091051ad,
0x0edd37c48a08a6d8,
          0x07e095624504536c,     0x8d70c431ac02a736,     0xc83862965601dd1b,
0x641c314b2b8ee083,
}

// iterationConstantsHex — константы итерации C[1]..C[12] из RFC 6986, п. 6.5,
// записанные в порядке стандарта (старшим байтом вперёд). В init они
// переворачиваются в рабочее представление состояния.
var iterationConstantsHex = [12]string{
 "b1085bda1ecadae9ebcb2f81c0657c1f2f6a76432e45d016714eb88d7585c4fc4b7ce09192676901a24
 "6fa3b58aa99d2f1a4fe39d460f70b5d7f3feea720a232b9861d55e0f16b501319ab5176b12d699585cb
 "f574dcac2bce2fc70a39fc286a3d843506f15e5f529c1f8bf2ea7514b1297b7bd3e20fe490359eb1c1c
 "ef1fdfb3e81566d2f948e1a05d71e4dd488e857e335c3c7d9d721cad685e353fa9d72c82ed03d675d8b
 "4bea6bacad4747999a3f410c6ca923637f151c1f1686104a359e35d7800fffbdbfcd1747253af5a3dff
 "ae4faeae1d3ad3d96fa4c33b7a3039c02d66c4f95142a46c187f9ab49af08ec6cffaa6b71c9ab7b40af
 "f4c70e16eeaac5ec51ac86febf240954399ec6c7e6bf87c9d3473e33197a93c90992abc52d822c37064
 "9b1f5b424d93c9a703e7aa020c6e41414eb7f8719c36de1e89b4443b4ddbc49af4892bcb929b069069d
 "378f5a541631229b944c9ad8ec165fde3a7d3a1b258942243cd955b7e00d0984800a440bdbb2ceb17b2
 "abbedea680056f52382ae548b2e4f3f38941e71cff8a78db1fffe18a1b3361039fe76702af69334b7a1
 "7bcd9ed0efc889fb3002c6cd635afe94d8fa6bbbebab07612001802114846679 8a1d71efea48b9caefb
 "378ee767f11631bad21380b00449b17acda43c32bcdf1d77f82012d430219f9b5d80ef9d1891cc86e71
}
```

```
// iterationConstants — те же константы в рабочем порядке байтов.
var iterationConstants [12][64]byte

func init() {
  for i, s := range iterationConstantsHex {
    raw, err := hex.DecodeString(s)
    if err != nil {
      panic("gost3411: некорректная константа C[%d]: " + err.Error())
    }
    iterationConstants[i] = stateFromBigEndian(raw)
  }
}

// stateFromBigEndian переставляет байты из записи стандарта (a_63 || ... || a_0)
// во внутреннее представление (a_0 в нулевом элементе).
func stateFromBigEndian(b []byte) [64]byte {
  var s [64]byte
  for i := 0; i < 64; i++ {
    s[i] = b[63-i]
  }
  return s
}

// blockFromMessage записывает 64 байта сообщения в состояние: первый байт
// блока становится младшим, как требует порядок обработки стандарта.
func blockFromMessage(b []byte) [64]byte {
  var s [64]byte
  copy(s[:], b)
  return s
}

// S — нелинейная подстановка: каждый байт вектора заменяется по таблице Pi.
func S(a [64]byte) [64]byte {
  var out [64]byte
  for i := 0; i < 64; i++ {
    out[i] = pi[a[i]]
  }
  return out
}

// P — перестановка байтов: новый байт i равен байту Tau(i) исходного вектора.
func P(a [64]byte) [64]byte {
  var out [64]byte
  for i := 0; i < 64; i++ {
    out[i] = a[tau[i]]
  }
  return out
}

// L — линейное преобразование. Вектор делится на восемь 64-битных частей,
// каждая умножается на матрицу A над GF(2):
//
//  c = b_63 * A[0] xor ... xor b_0 * A[63].
func L(a [64]byte) [64]byte {
  var out [64]byte
  for part := 0; part < 8; part++ {
    b := binary.LittleEndian.Uint64(a[part*8 : part*8+8])
    var c uint64
    for bit := 0; bit < 64; bit++ {
      if b&(1<<uint(bit)) != 0 {
        c ^= aMatrix[63-bit]
      }
    }
```

```
    binary.LittleEndian.PutUint64(out[part*8:part*8+8], c)
  }
  return out
}

// lps — композиция L(P(S(a))), основная перестановка раунда.
func lps(a [64]byte) [64]byte {
  return L(P(S(a)))
}

// X — сложение по модулю 2 (побитовое исключающее ИЛИ) двух векторов.
func X(a, b [64]byte) [64]byte {
  var out [64]byte
  for i := 0; i < 64; i++ {
    out[i] = a[i] ^ b[i]
  }
  return out
}

// keySchedule вычисляет K[i+1] = LPS(K[i] xor C[i]) для раунда i (1..12).
func keySchedule(k [64]byte, round int) [64]byte {
  return lps(X(k, iterationConstants[round-1]))
}

// E — раундовая функция: 12 раз применяет LPS и складывает результат
// с очередным ключом K[i], начиная с X[K[1]](m).
func E(k, m [64]byte) [64]byte {
  state := X(k, m)
  for round := 1; round <= 12; round++ {
    state = lps(state)
    k = keySchedule(k, round)
    state = X(state, k)
  }
  return state
}

// gN — функция сжатия g_N(h, m) = E(LPS(h xor N), m) xor h xor m.
func gN(h, m, n [64]byte) [64]byte {
  k := lps(X(h, n))
  t := E(k, m)
  return X(X(t, h), m)
}

// add512 складывает два 512-битных вектора по модулю 2^512.
func add512(a, b [64]byte) [64]byte {
  var out [64]byte
  carry := 0
  for i := 0; i < 64; i++ {
    sum := int(a[i]) + int(b[i]) + carry
    out[i] = byte(sum)
    carry = sum >> 8
  }
  return out
}

// addBits прибавляет к вектору число битов (N := N + |M| mod 2^512).
func addBits(a [64]byte, bits uint64) [64]byte {
  var b [64]byte
  binary.LittleEndian.PutUint64(b[:8], bits)
  return add512(a, b)
}

// gost3411 — реализация hash.Hash для ГОСТ Р 34.11-2012.
```

```
//
// Принятые через Write байты накапливаются: Sum вычисляет дайджест,
// не изменяя состояние.
type gost3411 struct {
  size    int
  iv      [64]byte
  message []byte
}

// NewGOST3411_512 возвращает хеш-функцию с 512-битным кодом (IV = 0^512).
func NewGOST3411_512() hash.Hash {
  return &gost3411{size: gost512Size}
}

// NewGOST3411_256 возвращает хеш-функцию с 256-битным кодом (IV =
(00000001)^64).
func NewGOST3411_256() hash.Hash {
  var iv [64]byte
  for i := range iv {
    iv[i] = 0x01
  }
  return &gost3411{size: gost256Size, iv: iv}
}

func (d *gost3411) Size() int      { return d.size }
func (d *gost3411) BlockSize() int { return gostBlockSize }

// Reset сбрасывает накопленное сообщение; начальное значение h не меняется.
func (d *gost3411) Reset() {
  d.message = d.message[:0]
}

// Write добавляет данные к сообщению. Ошибок быть не может.
func (d *gost3411) Write(p []byte) (int, error) {
  d.message = append(d.message, p...)
  return len(p), nil
}

// Sum дописывает к b дайджест, не изменяя состояние.
func (d *gost3411) Sum(b []byte) []byte {
  h := d.finish()
  // Дайджест выдаётся старшим байтом вперёд. Для 256-битного варианта
  // берётся MSB_256 — старшие 32 байта состояния.
  for i := 0; i < d.size; i++ {
    b = append(b, h[63-i])
  }
  return b
}

// finish выполняет шаги 2 и 3 стандарта: обработку полных блоков с конца,
// дополнение 0...01|M, затем две заключительные функции g_0.
func (d *gost3411) finish() [64]byte {
  h := d.iv
  var n, sigma [64]byte
  data := d.message

  // Шаг 2: пока в сообщении есть полный блок, берём очередные 512 бит.
  for len(data) >= gostBlockSize {
    block := blockFromMessage(data[:gostBlockSize])
    h = gN(h, block, n)
    n = addBits(n, 512)
    sigma = add512(sigma, block)
    data = data[gostBlockSize:]
```

```
  }

  // Шаг 3.1: m = 0^(511-|M|) || 1 || M.
  var last [64]byte
  copy(last[:], data)
  last[len(data)] = 0x01

  // Шаги 3.2–3.4.
  h = gN(h, last, n)
  n = addBits(n, uint64(len(data))*8)
  sigma = add512(sigma, last)

  // Шаги 3.5–3.6: h = g_0(h, N), h = g_0(h, Sigma).
  zero := [64]byte{}
  h = gN(h, n, zero)
  h = gN(h, sigma, zero)
  return h
}
```

Листинг А.2 – SHA-1

```
package hash

// Хеш-функция SHA-1 по FIPS 180-4.

import (
  "encoding/binary"
  "hash"
  "math/bits"
)

const (
  sha1BlockSize = 64
  sha1Size      = 20
)

// Четыре раундовые константы K_t из FIPS 180-4, п. 4.2.1.
var sha1Constants = [4]uint32{
  0x5a827999,
  0x6ed9eba1,
  0x8f1bbcdc,
  0xca62c1d6,
}

// sha1Digest — реализация hash.Hash для SHA-1.
type sha1Digest struct {
  state [5]uint32
  block [sha1BlockSize]byte
  used  int
  total uint64
}

// NewSHA1 возвращает хеш-функцию SHA-1.
func NewSHA1() hash.Hash {
  d := &sha1Digest{}
  d.Reset()
  return d
}

func (d *sha1Digest) Size() int      { return sha1Size }
func (d *sha1Digest) BlockSize() int { return sha1BlockSize }

// Reset возвращает буфер в начальное значение IV из FIPS 180-4, п. 5.3.1.
```

```
func (d *sha1Digest) Reset() {
  d.state = [5]uint32{
    0x67452301,
    0xefcdab89,
    0x98badcfe,
    0x10325476,
    0xc3d2e1f0,
  }
  d.used = 0
  d.total = 0
}

// Write добавляет данные, обрабатывая все накопленные полные блоки.
func (d *sha1Digest) Write(p []byte) (int, error) {
  n := len(p)
  d.total += uint64(n)

  for len(p) > 0 {
    copied := copy(d.block[d.used:], p)
    d.used += copied
    p = p[copied:]
    if d.used == sha1BlockSize {
      d.processBlock(d.block[:])
      d.used = 0
    }
  }
  return n, nil
}

// Sum дописывает дайджест к b, не изменяя состояние.
func (d *sha1Digest) Sum(b []byte) []byte {
  clone := *d
  clone.finish()
  var out [sha1Size]byte
  for i, v := range clone.state {
    binary.BigEndian.PutUint32(out[i*4:], v)
  }
  return append(b, out[:]...)
}

// finish дополняет сообщение по FIPS 180-4, п. 5.1.1: бит 1, нули и
// 64-битная длина сообщения в битах, после чего обрабатывает последний блок.
func (d *sha1Digest) finish() {
  bitLen := d.total * 8

  d.block[d.used] = 0x80
  d.used++

  if d.used > sha1BlockSize-8 {
    for d.used < sha1BlockSize {
      d.block[d.used] = 0
      d.used++
    }
    d.processBlock(d.block[:])
    d.used = 0
  }

  for d.used < sha1BlockSize-8 {
    d.block[d.used] = 0
    d.used++
  }
  binary.BigEndian.PutUint64(d.block[sha1BlockSize-8:], bitLen)
  d.processBlock(d.block[:])
```

```
}

// processBlock выполняет 80 раундов SHA-1 для одного 512-битного блока.
func (d *sha1Digest) processBlock(block []byte) {
  var w [80]uint32

  // Первые 16 слов берутся из блока напрямую.
  for i := 0; i < 16; i++ {
    w[i] = binary.BigEndian.Uint32(block[i*4 : i*4+4])
  }
  // Остальные: W[t] = rotl(W[t-16] xor W[t-14] xor W[t-8] xor W[t-3], 1).
  for t := 16; t < 80; t++ {
    w[t] = bits.RotateLeft32(w[t-16]^w[t-14]^w[t-8]^w[t-3], 1)
  }

  a, b, c, e, f := d.state[0], d.state[1], d.state[2], d.state[3], d.state[4]

  for t := 0; t < 80; t++ {
    var fn, k uint32
    switch {
    case t < 20:
      fn = (b & c) | (^b & e)
      k = sha1Constants[0]
    case t < 40:
      fn = b ^ c ^ e
      k = sha1Constants[1]
    case t < 60:
      fn = (b & c) | (b & e) | (c & e)
      k = sha1Constants[2]
    default:
      fn = b ^ c ^ e
      k = sha1Constants[3]
    }

    temp := bits.RotateLeft32(a, 5) + fn + f + k + w[t]
    f = e
    e = c
    c = bits.RotateLeft32(b, 30)
    b = a
    a = temp
  }

  d.state[0] += a
  d.state[1] += b
  d.state[2] += c
  d.state[3] += e
  d.state[4] += f
}
```